

🌟 DAY - 19: MASTER JavaScript Events 🌟

Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#) 📁 | [X: @_19_neeraj](#) 🐦 | [GitHub: Neeraj05042001](#) 🧑

📋 Table of Contents

1. 🌟 Introduction to Events
2. 🔄 Event Handling Methods
3. 🎯 Event Listeners
4. 🕒 DOM Content Loaded
5. 📄 Event Object
6. 🔄 Event Propagation
7. 👥 Event Delegation
8. 🚫 Preventing Default Behavior
9. 🎭 Custom Events
10. 💪 Practice Tasks
11. 🎤 Interview Questions
12. 📖 Quick Reference



🌟 Introduction to Events 🚀

What is an Event? 🤔

An event in JavaScript is an action that occurs in the browser, which can be triggered by the user or by the browser itself.

- 🖱️ A user clicking a button
- 🌐 A webpage finishing loading
- 📄 A form being submitted
- ⌨️ A key being pressed
- 🖱️ The mouse moving over an element

Events are the core of interactive web applications, allowing your code to respond to user actions and browser activities.

Common Types of Events:

- **Mouse Events:** `click`, `dblclick`, `mousedown`, `mouseup`, `mousemove`, `mouseover`, `mouseout`
- **Keyboard Events:** `keydown`, `keyup`, `keypress`
- **Form Events:** `submit`, `change`, `focus`, `blur`
- **Window Events:** `load`, `resize`, `scroll`, `unload`
- **Touch Events:** `touchstart`, `touchmove`, `touchend`

Why Events Matter?

Events allow your website to respond to user interactions, making your application dynamic and interactive. Without events, websites would be static and non-responsive.

💡 **Knowledge Check:** What are three common mouse events?

▶ Answer

Event Handling Methods

There are three main ways to handle events in JavaScript:

1. Event Handling in HTML Markup

You can add event handlers directly in HTML tags using attributes that start with "on" like `onclick`, `onmouseover`, etc.

```
<button id="myBtn" onClick="handleClick('Hello')">Click Me</button>
```

```
function handleClick(greeting) {  
  console.log(`Button Clicked with a ${greeting}`);  
}
```

Pros:

-  Simple and straightforward
-  Directly visible in HTML

Cons:

-  Mixes HTML and JavaScript (poor separation of concerns)
-  Limited functionality
-  Limited to one handler per event type

- ❌ Harder to maintain in larger applications
- ❌ Can become messy for complex interactions

2. Event Handling in JavaScript

You can assign event handlers using DOM properties in JavaScript:

```
const myBtn2Elem = document.getElementById("myBtn2");
myBtn2Elem.onclick = function() {
  console.log("My Button 2 Clicked");
}
```

Pros:

- ✅ Better separation of concerns
- ✅ More flexibility in handler assignment

Cons:

- ❌ Still limited to one handler per event type
- ❌ Previous handlers are overwritten:

```
// This second handler REPLACES the first one
myBtn2Elem.onclick = function() {
  console.log("My Button 2 Clicked Again");
}
```

 **Knowledge Check:** What's the main disadvantage of using the `onclick` property to handle events?

► Click for answer

 **Knowledge Check:** What happens if you assign multiple onclick handlers using the same property?

► Answer

3. Event Listeners

addEventListener() Method

The most powerful way to handle events is using the `addEventListener()` method:

```
element.addEventListener(eventType, handlerFunction, [options]);
```

Example:

```
const countBtn = document.getElementById("countBtn");
let counter = 0;

function handleCount() {
  console.log(counter);
  counter++;
}

countBtn.addEventListener("click", handleCount);
```

Pros:

-  Can add multiple listeners for the same event
-  More control over event handling
-  Clean separation of concerns
-  Better event handling options

removeEventListener() Method

To stop listening to an event, you can remove the event listener:

```
element.removeEventListener(eventType, handlerFunction);
```

Important:  The function reference must be the same one used in `addEventListener` !

```
// This works because we're using the same function reference
countBtn.removeEventListener("click", handleCount);

// This would NOT work because it's a new anonymous function
countBtn.removeEventListener("click", function() {
  console.log(counter);
  counter++;
});
```

Handling Multiple Listeners

With `addEventListener()` , you can attach multiple handlers to the same event/element:

```
const counBtnElem = document.getElementById("countBtn");
```

```
function handleCount() {
    console.log(counter);
    counter++;
}

function greetMe() {
    console.log("Thank You!");
}

// Both functions will run when button is clicked
counBtnElem.addEventListener("click", handleCount);
counBtnElem.addEventListener("click", greetMe);
```

Now when the button is clicked, both functions will execute in the order they were added.

 **Knowledge Check:** How would you correctly remove an event listener that was previously added?

► Click for answer

 **Knowledge Check:** Why won't this code successfully remove the event listener?

```
button.addEventListener("click", function() { alert("Clicked!"); });
button.removeEventListener("click", function() { alert("Clicked!"); });
```

► Answer

DOM Content Loaded

The `DOMContentLoaded` event fires when the HTML document has been completely loaded and parsed, without waiting for stylesheets, images, and subframes to finish loading:

Wrong way (won't work):

```
// This won't work (property approach)
document.onDOMContentLoaded = function() {
    console.log("DOM Content Loaded...");
};

**Correct way**:

// This will work (addEventListener approach)
document.addEventListener("DOMContentLoaded", function() {
    console.log("DOM Content Loaded...");
});
```

Why is this important? This ensures your JavaScript doesn't try to access DOM elements before they're available, preventing "null" or "undefined" errors.

When to use it:

- 🚀 When you need to manipulate DOM elements before the page is fully loaded
- ⚡ When you want your JavaScript to run as soon as the DOM is ready, without waiting for images and other resources

Difference from `window.onload` :

- 🔄 `DOMContentLoaded` fires when the DOM is ready
- 🌐 `window.onload` fires when everything (images, stylesheets, etc.) is loaded

💡 **Knowledge Check:** When does the `DOMContentLoaded` event fire compared to the `load` event?

▶ Click for answer

📄 Event Object 📦

When an event occurs, the browser creates an **event object** that contains details about the event. This object is automatically passed to your event handler function.

```
const searchElem = document.getElementById("search-id");
function handleChange(event) {
  console.log(event); // The entire event object
  console.log("Target:", event.target); // The element that triggered the event
  console.log("Target Name:", event.target.name); // Name attribute of the target
  console.log("Target Value:", event.target.value); // Current value of the target
  console.log("Event Type:", event.type); // Type of event (e.g., "change")
  console.log("Current Target:", event.currentTarget); // Element that the listener is attached to
  console.log("this:", this); // Same as currentTarget in regular function (not arrow function)
}

searchElem.addEventListener("change", handleChange);
```

Key Properties of the Event Object 🔑

Property	Description
<code>target</code>	🎯 The element that triggered the event
<code>currentTarget</code>	🔄 The element that the event listener is attached to

Property	Description
<code>type</code>	 The type of event (e.g., "click", "submit")
<code>preventDefault()</code>	 Method to prevent default browser behavior
<code>stopPropagation()</code>	 Method to stop event bubbling
<code>timeStamp</code>	 When the event occurred
<code>clientX/clientY</code>	 Mouse coordinates (for mouse events)
<code>key</code>	 Key pressed (for keyboard events)

 **Knowledge Check:** What's the difference between `event.target` and `event.currentTarget` ?

► [Click for answer](#)

Event Propagation

When an event occurs on an element, it doesn't just trigger handlers on that element. The event "propagates" through the DOM tree in two main phases:

Event Bubbling

By default, events "bubble" up from the target element to the root of the document.

Child Element → Parent → Grandparent → ... → document

Example:

```
<div id="outer">
  <div id="inner">
    <button id="button">Click Me</button>
  </div>
</div>
```

```
document.getElementById("button").addEventListener("click", function() {
  console.log("Button clicked");
});
```

```
document.getElementById("inner").addEventListener("click", function() {
  console.log("Inner div clicked");
});
```

```
document.getElementById("outer").addEventListener("click", function() {
```

```
console.log("Outer div clicked");
});
```

If you have a button inside a div and both have click handlers, clicking the button will:

1. First trigger the button's click handler
2. Then trigger the div's click handler

Clicking the button will log:

1. "Button clicked"
2. "Inner div clicked"
3. "Outer div clicked"

Event Capturing

The opposite of bubbling - events are first captured by the outermost element and then propagated to the inner elements.

```
document → ... → Grandparent → Parent → Target element
```

To use event capturing, set the third parameter of `addEventListener` to `true`:

```
element.addEventListener("click", handler, true); // Use capturing phase
```

Example with capturing:

```
document.getElementById("outer").addEventListener("click", function() {
  console.log("Outer div - CAPTURE phase");
}, true);

document.getElementById("inner").addEventListener("click", function() {
  console.log("Inner div - CAPTURE phase");
}, true);

document.getElementById("button").addEventListener("click", function() {
  console.log("Button - CAPTURE phase");
}, true);
```

Clicking the button with both sets of listeners would log:

1. "Outer div - CAPTURE phase"
2. "Inner div - CAPTURE phase"

3. "Button - CAPTURE phase"
4. "Button clicked" (bubbling)
5. "Inner div clicked" (bubbling)
6. "Outer div clicked" (bubbling)

Stop Propagation 🛑

To prevent an event from propagating further, use `event.stopPropagation()`, means to prevent an event from bubbling up (or capturing down):

```
element.addEventListener("click", function(event) {  
  event.stopPropagation(); // Stops the event from bubbling up  
  console.log("This handler will run, but the event won't propagate further");  
});
```

💡 **Knowledge Check:** In what order will these event handlers execute when the button is clicked?

```
<div id="outer">  
  <div id="inner">  
    <button id="btn">Click</button>  
  </div>  
</div>  
  
<script>  
  document.getElementById("outer").addEventListener("click", () => console.log("Outer"), true)  
  document.getElementById("inner").addEventListener("click", () => console.log("Inner"), false)  
  document.getElementById("btn").addEventListener("click", () => console.log("Button"), false)  
</script>
```

▶ Answer

💡 **Knowledge Check:** If you have event listeners on nested elements and want the event to only trigger on the most specific element, what method would you use?

▶ Click for answer

👥 Event Delegation 🧑‍🤝‍🧑

Event delegation is a technique where you attach a single event listener to a parent element instead of multiple listeners on child elements:

Without delegation (inefficient for many items):

```
// Assuming we have a list with many items
const items = document.querySelectorAll("li");
items.forEach(item => {
  item.addEventListener("click", function() {
    console.log("Item clicked:", this.textContent);
  });
});
```

With delegation (more efficient):

```
// Single listener on the parent
document.getElementById("list").addEventListener("click", function(event) {
  // Check if the clicked element is an li
  if (event.target.tagName === "LI") {
    console.log("Item clicked:", event.target.textContent);
  }
});
```

Benefits of Event Delegation 🌟

1. 🚀 **Memory Efficiency:** One listener instead of many
2. ✨ **Dynamic Elements:** Works with elements added after page load
3. 📄 **Less Code:** Simpler implementation for large numbers of similar elements

💡 **Knowledge Check:** Why is event delegation especially useful for dynamically created elements?

▶ Click for answer

💡 **Knowledge Check:** Why would event delegation be particularly useful for a to-do list application where items can be added or removed?

▶ Answer

🚫 Preventing Default Behavior

Many events have default actions associated with them. For example:

- 📄 Clicking a link navigates to its URL
- 📄 Submitting a form sends the data and reloads the page
- ✅ Clicking a checkbox toggles its state

To prevent these default behaviors, use `event.preventDefault()` :

```
// Prevent a link from navigating to its URL
document.getElementById("websiteLink").addEventListener("click", function(event) {
    event.preventDefault();
    console.log("Default link behavior prevented!");
});

// Prevent a form from submitting and reloading the page
document.getElementById("loginForm").addEventListener("submit", function(event) {
    event.preventDefault();
    console.log("Form submission prevented!");
    // Here you could validate the form or submit via AJAX instead
});
```

Common use cases:

-  Custom form validation before submission
-  AJAX form submissions without page reload
-  Custom navigation behavior for links
-  Creating custom UI controls

 **Knowledge Check:** In a form submit event handler, what method should you call if you want to handle the form submission with JavaScript instead of letting the browser submit it?

► Click for answer

 **Knowledge Check:** What would happen if you removed the `preventDefault()` call from a form submit handler?

► Answer

Custom Events

Custom events allow you to create your own event types for component communication.

Creating and Dispatching Custom Events:

```
// Step 1: Create a CustomEvent
const myEvent = new CustomEvent("userLoggedIn", {
  detail: {
    username: "tapascript",
    role: "admin",
  },
});
```

```
// Step 2: Listen for the Custom Event
document.addEventListener("userLoggedIn", (event) => {
    console.log("User login detected:", event.detail.username);
});

// Step 3: Dispatch the Custom Event
document.dispatchEvent(myEvent);
```

Practical Example - Module Communication:

```
// Module 1: Authentication logic
function loginUser(username) {
    const event = new CustomEvent("userLoggedIn", {
        detail: { username },
    });
    document.dispatchEvent(event);
}

// Module 2: Navbar or Sidebar
document.addEventListener("userLoggedIn", (event) => {
    const user = event.detail.username;
    document.getElementById("welcome").textContent = `Welcome, ${user}!`;
});

// Later, when user logs in
loginUser("tapascript");
```

Benefits of Custom Events:

1.  **Loose Coupling:** Components don't need direct references to each other
2.  **Maintainability:** Easier to maintain and test components in isolation
3.  **Scalability:** Makes it easier to add new components that respond to the same events

 **Knowledge Check:** How do you pass data with a custom event?

► Click for answer

Practice Tasks

1.  **Basic Event Handler:** Create a button that changes the background color of the page when clicked.
2.  **Multiple Event Listeners:** Add two different event listeners to a single button - one that changes text color and another that changes font size.

3. 🔍 **Event Object Exploration:** Create a text input that logs the key code whenever a key is pressed, using the event object.
 4. 🔄 **Bubbling Demo:** Create a nested structure (div > div > button) where each element has a click handler. Observe how events bubble through all elements.
 5. 🛑 **Stop Propagation:** Modify the above example to prevent the click event from reaching the outer div.
 6. 👤 **Event Delegation:** Create a to-do list where you can add new items. Use event delegation to handle clicks on the "delete" button for each item.
 7. 🚫 **Prevent Default:** Create a form that normally submits to a different page, but prevent its default behavior and log the form data to the console instead.
 8. 🎭 **Custom Event:** Create a custom "theme-changed" event that gets dispatched when the user selects a different theme. Have multiple components listen for and respond to this event.
 9. 📱 **Mobile Events:** Create an element that responds to both click events (on desktop) and touch events (on mobile).
 10. 🏆 **Advanced Challenge:** Create a simple drag-and-drop interface using mousedown, mousemove, and mouseup events.
-

🎤 Interview Questions 🎯

Q1: What is event bubbling and event capturing in JavaScript? How do they differ?

► Answer

Q2: Explain the difference between `event.preventDefault()` and `event.stopPropagation()`.

► Answer

Q3: What is event delegation and why is it useful?

► Answer

Q4: How would you create and use a custom event in JavaScript?

► Answer

Q5: What's the difference between the "load" and "DOMContentLoaded" events?

► Answer

Q6: How would you properly remove an event listener?

► Answer

Q7: What is the difference between event.target and event.currentTarget?

► Answer

Q8: How can you prevent a form from submitting using JavaScript?

► Answer

Q9: What's the difference between the three ways of handling events in JavaScript?

► Answer

Q10: How does the "this" keyword work in event handlers?

► Answer

Quick Revision ⚡

Event Types 🎯

Event Type	Triggered When
click	 Element is clicked
submit	 Form is submitted
change	 Form element value changes
input	 Value of input/textarea changes
keydown / keyup	 Keyboard key is pressed/released
mouseover / mouseout	 Mouse enters/leaves element
load	 Page and resources are fully loaded

Event Type	Triggered When
<code>DOMContentLoaded</code>	 DOM is fully loaded
<code>focus</code> / <code>blur</code>	 Element gains/loses focus
<code>resize</code>	 Window is resized

Event Handling Methods

Method	Syntax	Multiple Handlers	Notes
HTML Attributes	<code><button onclick="fn()"></code>	 No	Mixes HTML and JS
DOM Properties	<code>element.onclick = fn</code>	 No	Overwrites previous
<code>addEventListener</code>	<code>element.addEventListener("click", fn)</code>	 Yes	Most flexible

Event Object Properties

Property	Purpose
<code>event.target</code>	 Element that triggered the event
<code>event.currentTarget</code>	 Element with the attached listener
<code>event.type</code>	 Type of event (e.g., "click")
<code>event.preventDefault()</code>	 Prevents default browser behavior
<code>event.stopPropagation()</code>	 Stops event bubbling

Event Flow Summary

1. Capturing Phase : Root → Target (with `addEventListener(..., true)`)
2. Target Phase : The element itself
3. Bubbling Phase : Target → Root (default behavior)

🌟 Best Practices ¹⁰⁰

- 📌 Use `addEventListener` for most event handling
 - 📁 Use event delegation for lists and similar structures
 - 💾 Store function references when you need to remove listeners
 - 🕒 Always consider event bubbling when debugging
 - 🧩 Use custom events for modular code architecture
 - ⚡ Prefer `DOMContentLoaded` over `window.load` when possible
-

🚀 *Let's Connect & Level Up Together*

Follow me for daily JavaScript tips, insightful notes, and project-based learning.

 [X \(Twitter\)](#)

 [LinkedIn](#)

 [Telegram](#)

 [Threads](#)

 Dive into the full notes on GitHub → [40 Days JavaScript](#)

© 2025 • Crafted with ❤️ by Neeraj

 *Next: Day - 20: Advanced DOM Tips and Tricks* 

🚀 Happy JavaScript coding! 🚀